

Teoría de Conjuntos

1.1 Introducción: representación, pertenencia y operaciones Puente entre Matemática y Metodología de la Programación

Mentorías - Maria Julia Pareja Abarca

Universidad Católica de Santa María
Escuela Profesional de Ingeniería de Sistemas

Semestre I · 2026

Objetivo de la sesión

Al finalizar la sesión (2 horas) el estudiante podrá:

- Representar conjuntos por extensión, comprensión y diagramas de Venn.
- Determinar relaciones de pertenencia y contención.
- Aplicar las operaciones fundamentales entre conjuntos.
- Trasladar estos conceptos a Python: secuencias (`range`, `list`, `tuple`) y el tipo `set`.
- Verificar programáticamente propiedades de relaciones binarias.

Agenda

- 1 Fundamentos de la Teoría de Conjuntos
- 2 Operaciones entre Conjuntos
- 3 Secuencias y Conjuntos en Python
- 4 Conjuntos orientados a relaciones
- 5 Cierre

Distribución de tiempo — Bloques 1 y 2

Tiempo	Actividad	Modalidad
0:00–0:10	Apertura: ¿por qué importan los conjuntos al programar?	Magistral
0:10–0:30	1.1 Representación: extensión, comprensión, Venn, pertenencia	Magistral
0:30–0:45	Subconjuntos, vacío, conjunto potencia $P(A)$	Magistral
0:45–1:05	Operaciones: unión, intersección, diferencia, complemento	Casos

Distribución de tiempo — Bloques 3 y 4

Tiempo	Actividad	Modalidad
1:05–1:15	Receso	—
1:15–1:35	Secuencias en Python: <code>range</code> , <code>list</code> , <code>tuple</code> vs. <code>set</code>	Taller
1:35–1:55	Relaciones: reflexiva, simétrica, antisimétrica, transitiva	Taller
1:55–2:00	Cierre, ejercicio integrador y asignación de práctica	Plenaria

¿Qué es un conjunto?

Un **conjunto** es una colección *bien definida* de objetos, llamados **elementos** o **miembros**, que comparten una propiedad común.

- “Bien definida” significa que, dado cualquier objeto, siempre se puede determinar sin ambigüedad si pertenece o no al conjunto.
- Fuente: Jiménez Murillo (2015), *Matemáticas para la computación*.

Notación básica

- Los conjuntos se nombran con letras **mayúsculas**: $A, B, C, \dots$
- Los elementos se nombran con letras **minúsculas**: $a, b, c, \dots$
- Los elementos se encierran entre llaves $\{ \}$.

$A = \{1, 2, 3\}$ — A es un conjunto con tres elementos.

Representación por extensión

Se listan **todos** los elementos del conjunto, separados por comas.

```
A = {1, 2, 3, 4, 5}
```

```
B = {rojo, verde, azul}
```

```
C = {a, e, i, o, u}           # vocales del espanol
```

Representación por comprensión

Se describe una **propiedad** que cumplen exclusivamente los elementos.

$A = \{ x \mid P(x) \}$ se lee: “el conjunto de los x tales que x cumple P ”

Comprensión — ejemplos

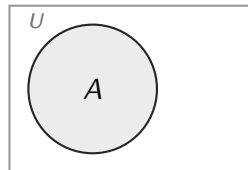
$$A = \{x \mid x \in \mathbb{N}, x < 6\} \rightarrow A = \{0, 1, 2, 3, 4, 5\}$$

$$B = \{x \in \mathbb{Z} \mid x^2 = 9\} \rightarrow B = \{-3, 3\}$$

$$P = \{x \in \mathbb{N} \mid x \text{ es primo}, x < 20\} \rightarrow \{2, 3, 5, 7, 11, 13, 17, 19\}$$

Diagramas de Venn

- Representación gráfica de un conjunto como región cerrada dentro de un rectángulo (el universo U).
- Es la forma más intuitiva para visualizar operaciones entre conjuntos (Bloque 2).



Relación de pertenencia

$x \in A$ se lee “ x pertenece a A ”

$x \notin A$ se lee “ x no pertenece a A ”

Ejemplo: $A = \{2, 4, 6, 8\}$

- $2 \in A$ (verdadero)
- $5 \notin A$ (falso que $5 \in A$)

Conjuntos especiales (I)

Nombre	Notación	Descripción
Conjunto vacío	$\emptyset$ ó $\{\}$	No contiene ningún elemento
Conjunto universal	U	Contiene todos los elementos del contexto
Conjunto unitario	$\{a\}$	Tiene exactamente un elemento

Conjuntos especiales (II)

Nombre	Notación	Descripción
Conjunto finito	$ A = n$	Número contable de elementos (cardinalidad n)
Conjunto infinito	$\mathbb{N}, \mathbb{Z}, \mathbb{R}$	No tiene un número finito de elementos

Subconjuntos

$A \subseteq B$ (subconjunto de B) $\iff$ todo elemento de A está también en B

$A \subset B$ (subconjunto *propio*) $\iff A \subseteq B$ y $A \neq B$

Conjunto potencia $P(A)$

$P(A)$: conjunto de **todos** los subconjuntos de A (incluye $\emptyset$ y A).

Si $|A| = n$, entonces $|P(A)| = 2^n$

Ejemplo: $A = \{1, 2\}$

$$P(A) = \{ \emptyset, \{1\}, \{2\}, \{1, 2\} \} \quad |P(A)| = 2^2 = 4$$

Operaciones entre conjuntos

Sean A y B subconjuntos de un universo U . Fuente: Alfaomega, *Matemáticas discretas: aplicaciones y ejercicios*.

- Unión, intersección, diferencia
- Complemento, diferencia simétrica
- Producto cartesiano, Leyes de De Morgan

Unión ($A \cup B$)

$$A \cup B = \{x \mid x \in A \vee x \in B\}$$

Ejemplo: $A = \{1, 2, 3\}$, $B = \{3, 4, 5\}$ $\rightarrow A \cup B = \{1, 2, 3, 4, 5\}$

Intersección ($A \cap B$)

$$A \cap B = \{x \mid x \in A \wedge x \in B\}$$

Ejemplo: $A = \{1, 2, 3\}$, $B = \{3, 4, 5\} \rightarrow A \cap B = \{3\}$

Diferencia ($A - B$)

$$A - B = \{x \mid x \in A \wedge x \notin B\}$$

Ejemplo: $A = \{1, 2, 3\}$, $B = \{3, 4, 5\}$

$$A - B = \{1, 2\} \qquad B - A = \{4, 5\}$$

Complemento (A' o $\bar{A}$)

$$A' = \{x \in U \mid x \notin A\} = U - A$$

Ejemplo: $U = \{1, \dots, 10\}$, $A = \{2, 4, 6, 8, 10\}$ $\rightarrow$ $A' = \{1, 3, 5, 7, 9\}$

Diferencia simétrica ($A\Delta B$)

$$A\Delta B = (A \cup B) - (A \cap B) = (A - B) \cup (B - A)$$

Ejemplo: $A = \{1, 2, 3\}$, $B = \{3, 4, 5\}$ $\rightarrow$ $A\Delta B = \{1, 2, 4, 5\}$

Producto cartesiano ($A \times B$)

$$A \times B = \{ (a, b) \mid a \in A \wedge b \in B \} \quad (\text{pares ordenados})$$

Ejemplo: $A = \{1, 2\}$, $B = \{x, y\}$

$$A \times B = \{(1, x), (1, y), (2, x), (2, y)\} \quad |A \times B| = |A| \cdot |B| = 4$$

Producto cartesiano — importancia

Este concepto es la **base formal de las relaciones** (Bloque 4): toda relación R entre A y B es un subconjunto de $A \times B$.

Leyes de De Morgan

$$(A \cup B)' = A' \cap B' \qquad (A \cap B)' = A' \cup B'$$

Base de la simplificación lógica; en Python:

```
not (a or b) == (not a) and (not b)
```

Secuencias vs. conjuntos en Python

Python distingue entre:

- **Secuencias** (ordenadas, permiten duplicados, indexables): `range`, `list`, `tuple`
- **Conjuntos** (no ordenados, sin duplicados): `set`

Tabla comparativa

Tipo	Ordenado / indexable	Mutable	Uso típico
range(n)	Sí	No	Secuencias aritméticas, contadores
list	Sí	Sí	Colecciones que cambian de tamaño/valor
tuple	Sí	No	Datos fijos, pares/registros
set	No	Sí (frozenset no)	Pertenencia rápida, álgebra de conjuntos

Indexación: secuencias vs. sets

Recordando la práctica de índices (Experiencia N.º 01): una lista/tupla a se recorre con $a[i]$, $i = 0, \dots, n - 1$, $n = \text{len}(a)$.

```
secuencia = ['Carlo', 'Jose', 'Luis']  
print(secuencia[0])           # 'Carlo' -> si funciona  
  
conjunto = {'Carlo', 'Jose', 'Luis'}  
print(conjunto[0])           # TypeError: not subscriptable
```

El tipo set y sus operaciones

```
A = {1, 2, 3}
B = {3, 4, 5}

print(A | B)      # union()          -> {1, 2, 3, 4, 5}
print(A & B)      # intersection()   -> {3}
print(A - B)      # difference()     -> {1, 2}
print(A ^ B)      # symmetric_difference() -> {1, 2, 4, 5}
```

Eliminar duplicados con set

```
nombres = ['Ana', 'Luis', 'Ana', 'Eva']  
unicos  = set(nombres)           # {'Ana', 'Luis', 'Eva'}
```

set/frozenset son la representación computacional más directa de un conjunto matemático; list/tuple corresponden a **sucesiones** (secuencias con índice).

Relación binaria: definición formal

Una relación R de A en B es cualquier subconjunto del producto cartesiano: $R \subseteq A \times B$

Si $A = B$, se dice que R es una relación definida **sobre** A .

Representación de una relación en Python

```
A = {1, 2, 3, 4}
# R: "x divide a y"
R = {(a, b) for a in A for b in A if b % a == 0}
print(R)
# {(1, 1), (1, 2), (1, 3), (1, 4), (2, 2), (2, 4), (3, 3), (4, 4)}
```

Propiedad: Reflexiva

$$\forall a \in A : (a, a) \in R$$

Todo elemento se relaciona consigo mismo.

Reflexiva — verificación en Python

```
def es_reflexiva(R, A):  
    return all((a, a) in R for a in A)
```

Propiedad: Simétrica

$$(a, b) \in R \rightarrow (b, a) \in R$$

Si a se relaciona con b , entonces b se relaciona con a .

Simétrica — verificación en Python

```
def es_simetrica(R):  
    return all((b, a) in R for (a, b) in R)
```

Propiedad: Antisimétrica

$$(a, b) \in R \wedge (b, a) \in R \rightarrow a = b$$

Nunca hay ida y vuelta entre elementos distintos.

Antisimétrica — verificación en Python

```
def es_antisimetrica(R):  
    return all(a == b for (a, b) in R if (b, a) in R)
```

Propiedad: Transitiva

$$(a, b) \in R \wedge (b, c) \in R \rightarrow (a, c) \in R$$

Las relaciones se “encadenan” correctamente.

Transitiva — verificación en Python

```
def es_transitiva(R):  
    return all((a, c) in R  
               for (a, b1) in R  
               for (b2, c) in R if b1 == b2)
```

Conjunción de propiedades: orden parcial

Una relación R sobre A es un **orden parcial** cuando cumple, a la vez (conjunción lógica $\wedge$):

reflexiva $\wedge$ antisimétrica $\wedge$ transitiva

Ejemplo: la relación “divide a” sobre $\mathbb{Z}^+$, o $\subseteq$ sobre $P(A)$.

Orden parcial — verificación en Python

```
def es_orden_parcial(R, A):  
    return (es_reflexiva(R, A) and  
            es_antisimetrica(R) and  
            es_transitiva(R))  
  
A = {1, 2, 3, 4}  
R = {(a, b) for a in A for b in A if b % a == 0}  
print(es_orden_parcial(R, A))          # True
```

Relación inversa

$$R^{-1} = \{ (b, a) \mid (a, b) \in R \}$$

Se invierte el orden de cada par ordenado.

Relación inversa — verificación en Python

```
def relacion_inversa(R):  
    return {(b, a) for (a, b) in R}  
  
R = {(1, 2), (2, 3), (1, 3)}  
print(relacion_inversa(R))    # {(2, 1), (3, 2), (3, 1)}
```

Ejercicio integrador de cierre

Sucesión $a = \text{'Carlo Jose Luis'}$ (Experiencia de Práctica N.º 01) convertida en lista: $a = [\text{'Carlo'}, \text{'Jose'}, \text{'Luis'}]$

Ejercicio integrador — parte 1

1. Construye el conjunto de índices válidos $I = \{0, 1, 2\}$ y verifica, usando pertenencia ($\in$), si $i = 5$ y $i = -1$ son índices válidos según $i = 0, \dots, n - 1$.

Ejercicio integrador — parte 2

2. Define la relación R sobre a como pares $(palabra_i, palabra_j)$ tales que $\text{len}(palabra_i) \leq \text{len}(palabra_j)$. Represéntala como conjunto de tuplas.

Ejercicio integrador — parte 3

3. Usando las funciones vistas, determina si R es reflexiva y si es transitiva. Conjetura qué tipo de orden forma R .

Síntesis de la sesión

- Un conjunto se representa por extensión, comprensión o Venn.
- Las operaciones básicas ($\cup, \cap, -, ', \Delta, \times$) construyen nuevos conjuntos.
- Python separa **secuencias** (indexables, con orden) de **conjuntos** (set, sin orden).
- Toda relación es un subconjunto de $A \times B$; sus propiedades (reflexiva, simétrica, antisimétrica, transitiva) se verifican con código simple.
- Reflexiva + antisimétrica + transitiva = **orden parcial**.

Referencias bibliográficas

- Jiménez Murillo, J. A. (2015). *Matemáticas para la computación* (2.º ed.). Alfaomega.
- Alfaomega. *Matemáticas discretas: aplicaciones y ejercicios*.
- Gómez Jiménez, E. y Aguilera Jinesta, R. (2022). *Metodología de la programación*. Alpha Editorial. [UCSM: 005.1.GOME.00]
- Joyanes Aguilar, L. (2020). *Fundamentos de programación*. [UCSM: 9786071514684]
- Sílabo 7101245 — Metodología de la Programación, EPIS UCSM, 2026.
- Sílabo 7101331 — Matemática, EPIS UCSM, 2026.

Gracias

Mentorías

UCSM · Escuela Profesional de Ingeniería de Sistemas